SAFE-Drive: AI-Assisted Traffic Monitoring and Road-Safety Review

Final implementation and verification report

Date: 28 October 2026

Repository: github.com/bhuvzzs24-spec/SAFE-Drive

Abstract

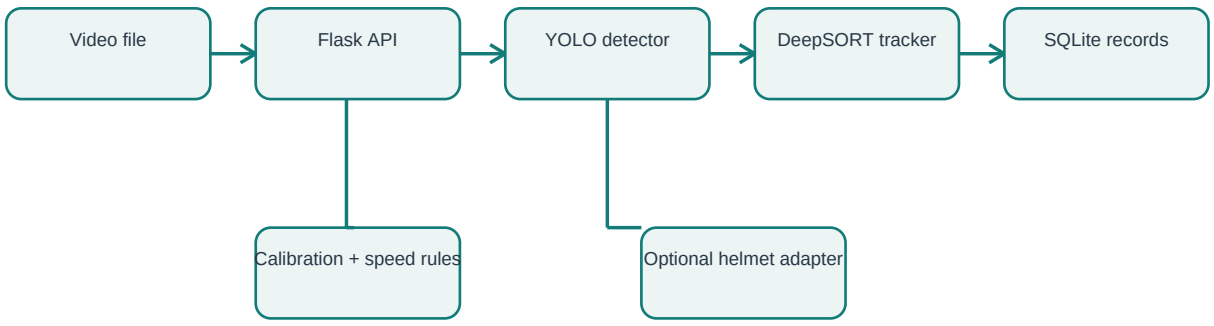
SAFE-Drive is a Flask web application for reviewing traffic video with an object-detection and multi-object tracking pipeline. It is designed to present detected road users, track identities, calibrated speed measurements, rule-based speed events, and reviewer decisions. The repository includes nine connected web pages, a SQLite-backed application, and optional integration points for a trained helmet classifier. This report separates implemented code from functionality that could not be exercised because no compatible model weights, traffic video, camera calibration, or live camera were available. No accuracy or performance metrics are claimed.

Problem and objectives

Road-safety video review can benefit from a repeatable workflow that associates detections across frames and records reviewable events. SAFE-Drive aims to provide that workflow while keeping model outputs and measured speeds tied to actual inputs. Its objectives are to ingest video, detect supported object classes, track them over time, estimate speed only with calibration, persist results, and provide an interface for analysis and review.

The current scope is offline video analysis and result review. The application is not a validated safety-critical system and does not provide genuine accident detection.

Architecture and data flow



Annotated results and review data are presented by the web interface.

The Flask application serves the HTML templates, CSS, JavaScript, and JSON endpoints. The analysis pipeline processes frames, associates detections with persistent tracker IDs, optionally derives calibrated speeds, records events, and produces annotated output when its model and runtime dependencies are available. SQLite stores session metadata, tracked objects, events, and review decisions. The browser uses API data; it must not invent

detection counts or speeds for an empty or unprocessed session.

Technologies

- Python and Flask for the web application and API.
- Ultralytics YOLO interface for object detection; compatible local weights are required.
- DeepSORT tracking for persistent object identities.
- OpenCV-based video handling and annotation.
- SQLite persistence.
- HTML, CSS, and JavaScript for the nine-page interface.
- The project dependency manifest and `.venv` provide the Python environment; see the repository for the authoritative package versions.

Detection and tracking

The detector class names are mapped to road-user categories, including supported vehicle categories and pedestrians. Detections are passed through the tracker to obtain track IDs and per-object histories. Pedestrians are tracked as tracked objects but are excluded from vehicle speed estimation. An annotated video can display actual detection boxes, labels, and IDs when a real analysis completes.

The code path is implemented, but no suitable YOLO weight file or traffic sample was available in the inspected project assets. Consequently, this sprint did not run a real video through the detector and tracker, did not generate an annotated sample, and did not verify detections visually. Model importability or passing unit tests is not evidence of successful inference.

Speed estimation and speed rules

Speed in real-world units requires valid road-plane calibration. The estimator uses the configured mapping from image coordinates to real-world distance and elapsed frame time. With no calibration, km/h is unavailable; the application must not substitute pixel motion or fabricated values. Pedestrians are not assigned vehicle speeds.

A controlled unit test checks a known-distance example: 10 metres traversed in 1 second corresponds to 36 km/h. This verifies the conversion logic, not camera calibration accuracy or a measured road vehicle. Configurable speed rules use confirmation/deduplication logic and can persist a violation event when the conditions are satisfied. The tested persistence workflow covers a generated speed-rule event and reviewer decision; no real footage speed violation was observed in this sprint.

Optional helmet classification

The optional helmet adapter loads only a user-supplied local model path (SAFE_DRIVE_HELMET_MODEL) and checks that the model has the expected class labels. It associates a helmet detection with a pedestrian upper-body region; ambiguous association is represented as uncertain. The adapter and persistence field are implemented, but no compatible trained half-helmet weights were found, so inference was not verified. The feature is experimental and does not implement temporal confirmation or a complete helmet-violation event workflow.

To enable inference, provide a compatible trained weights file and its documented class names, set SAFE_DRIVE_HELMET_MODEL to the local path, and validate it on representative labelled footage. A training dataset, validation split, and evaluation procedure are required before reporting model quality.

Accident detection and risk screening

Genuine accident detection is not implemented. The existing approach-risk/near-miss screening signals must not be presented as confirmed accidents. Speed-rule alerts are a separate event type. SAFE-Drive currently generates no accident alerts. A validated incident method, labelled incident and non-incident video, and evaluation against false positives and missed events are needed before this feature can be claimed.

Database and persistence

SQLite stores analysis sessions, generic tracked objects (including vehicle and pedestrian classes), compatibility vehicle-track records, optional helmet classification, events, and reviewer decisions. The persistence test closes and reopens the database and checks that a session, car and pedestrian tracks, helmet-classification value, generated speed-rule event, and review decision remain available. This verifies database persistence for the test fixture; it does not represent results from an actual analysed traffic video.

Website and workflow

The application provides nine connected pages: Overview, Video Analysis, Vehicle Tracking, Helmet Compliance, Speed Violations, Event Review, Analytics, Camera Management, and Settings. The Video Analysis page is the upload and processing workspace. Results should appear only after analysis and should reflect backend data. Vehicle Tracking describes IDs as object tracks, rather than claiming that an ID identifies a unique real-world vehicle across sessions. Helmet and event pages state the unavailable capabilities clearly.

During this sprint, the existing layout was preserved. Small frontend integration changes expose whether a helmet weights file is configured, show a helmet classification when the backend supplies one, and distinguish approach-risk screening from accident detection. The restarted Flask app loaded the current code. In the browser, all nine navigation destinations changed to their expected routes; CSS was visibly applied (computed body font was Inter/Avenir Next and the sidebar had its dark surface color). /, /static/app.css, /static/app.js, /api/sessions, and /api/analytics returned HTTP 200 in the Flask request log. The upload form accepted the listed video formats and displayed a validation message when analysis was started without a file. A real video upload and analysis could not be tested because no sample video or detector weights were available. Browser console inspection showed no errors during the checked navigation.

Setup and execution

From the repository root, use the existing project environment:

Open http://127.0.0.1:5170/. Install dependencies from the project's dependency file into the existing environment if needed. Do not place private CCTV footage, credentials, generated databases, virtual environments, or model weights in the public repository. Configure detector weights and camera calibration using the project-supported settings before expecting inference or real-world speed values.

Verification results

The current complete test run executed PYTHONPATH=src .venv/bin/python -m unittest discover -s tests -v and passed 31 tests with no failures: Ran 31 tests in 362.350s, OK. Coverage included API/database and pipeline logic, controlled speed conversion, track persistence, optional helmet handling, and review/event persistence. It did not include end-to-end inference on a real video because the required assets were absent. The previous recorded run had also passed 31 tests in 185.776 seconds.

For this sprint, node --check static/app.js and git diff --check returned success. Browser checks of navigation, rendered capability messages, asset loading, and no-file validation passed. The automated suite does not exercise real model inference, upload-to-video completion, camera streaming, or responsive viewport behavior.

No test was run for model accuracy, throughput, accident detection, production camera streaming, or deployment. No screenshot or annotated-video figure is included because no real video inference was available to produce one.

Limitations and required assets

The following items remain necessary for end-to-end validation:

1. A compatible YOLO detector weights file supporting the target vehicle and pedestrian classes.
2. A short, legally usable traffic video with adequate resolution, representative road users, and known metadata.
3. Camera calibration data or measured road geometry to map image coordinates to metres and validate km/h.
4. For helmet classification, a compatible half-helmet model with class names and a labelled validation set.
5. For accident detection, an implemented and evaluated incident method plus labelled incident and non-incident clips.
6. A live camera URL and access credentials only if camera integration is to be tested; never commit credentials.

No model metrics, detection counts, speed observations, incident outcomes, or screenshots are claimed in this report.

Reproducibility and publication

Repository: github.com/bhuvzzs24-spec/SAFE-Drive

Run the automated tests with:

Record the exact weights, video hash, calibration parameters, software versions, and command used for any future experiment. Report precision/recall or other metrics only after evaluation on a held-out labelled dataset. The project should be deployed publicly only after authentication, access control, host-compatible persistent storage, and production server configuration are in place; no public deployment is claimed here.